

Understanding Dependency Injection Lifetimes in .NET

Q: Different Service Lifetimes in Dependency Injection

In .NET, the Dependency Injection (DI) container provides three primary service lifetimes. Understanding these lifetimes is critical for managing memory and ensuring that objects are reused or created as required.

Singleton: A single instance of the service is created and shared throughout the entire lifetime of the application.

```
builder.Services.AddSingleton<INotificationService, EmailNotificationService>();
```

When to Use: When your service is stateless or you want the same instance to be used globally across the application.

Scoped: A new instance of the service is created per HTTP request.

```
builder.Services.AddScoped<INotificationService, EmailNotificationService>();
```

When to Use: When you need to maintain some form of state across a single request but want isolation between requests.

Transient: A new instance of the service is created every time it is requested.

```
builder.Services.AddTransient<INotificationService, EmailNotificationService>();
```

When to Use: When the service is stateless and you want a new instance every time to ensure no residual state is carried over between uses.

A: Detailed explanation with examples, use cases, and visual analogies for Singleton, Scoped, and Transient services.

Q: Explain with more example and cases

A: Expanded explanation including real-world examples like caching, DbContext, validators, etc. Also includes comparison tables, a testable sample project structure, and a visual analogy. Describes how to use Swagger/Postman to observe DI behavior and includes a test controller example.

Q: Yes (Request for visual + sample project + test setup)

A: Provided a visual diagram, code snippets for services, controller setup, DI configuration, and instructions on testing using Swagger/Postman.

Q: Create PDF of complete session with question asked

A: Generating the requested PDF summarizing the session.